Fundamentos de Complexidade de Algoritmos

Prof. Ricardo Mesquita

Complexidade e Desempenho de Algoritmos

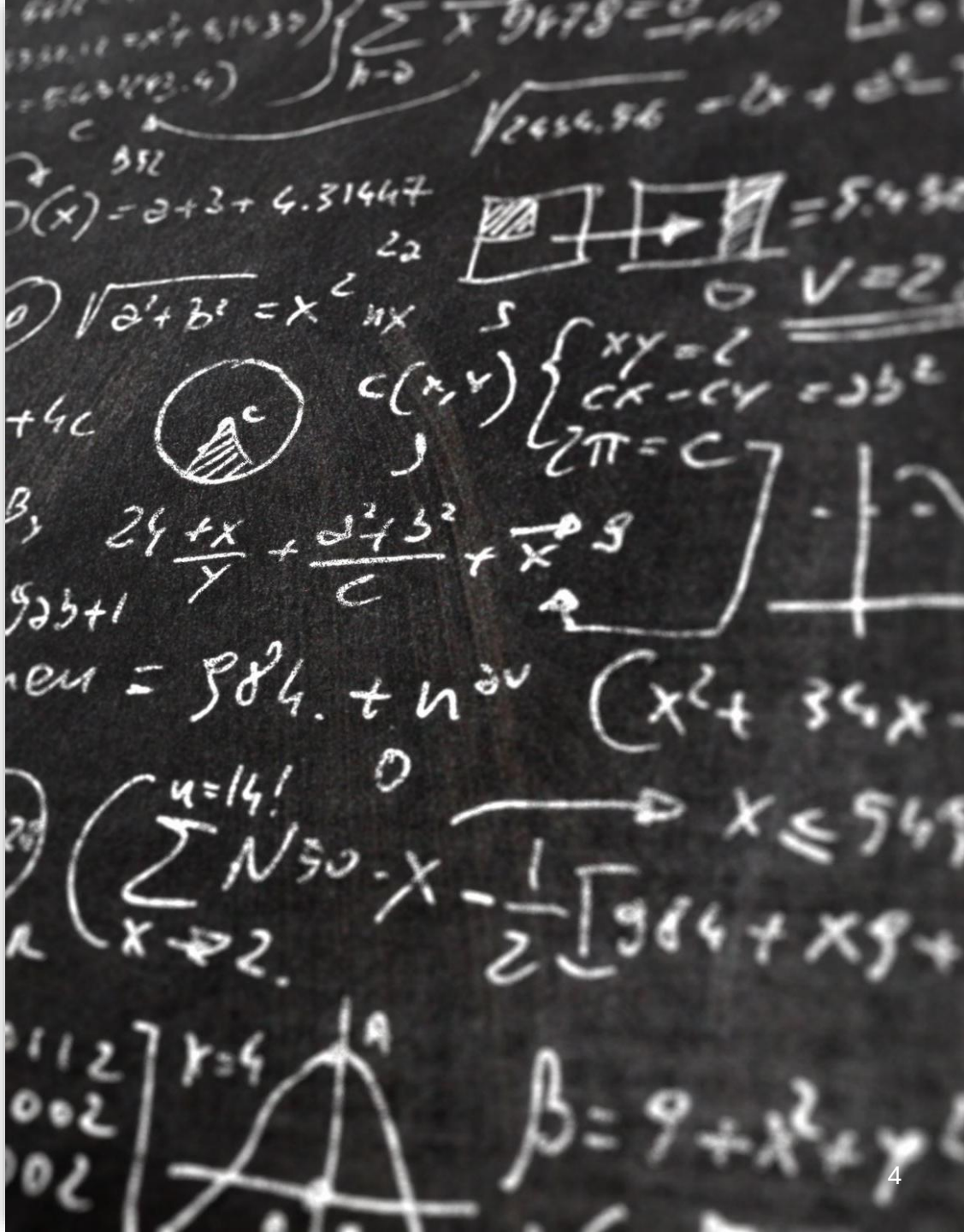
- A complexidade de um algoritmo reflete ***o esforço computacional requerido para executá-lo***.
- Esse esforço computacional mede a ***quantidade de trabalho***, em termos de ***tempo de execução*** ou da ***quantidade de memória*** requerida.
- ***Note***: as principais medidas de complexidade são ***tempo*** e ***espaço***, relacionadas à velocidade e à quantidade de memória, respectivamente, para a execução de um algoritmo.

Complexidade e Desempenho de Algoritmos

- Apesar de a comparação de programas reais, rodando em computadores reais, ser uma fonte de informação importante, os resultados são inevitavelmente afetados pela programação e pelas características das máquinas.
- Uma alternativa útil vem de se fazer ***uma análise matemática do algoritmo*** de maneira geral.
  - Essa análise, além de ser independente da implementação, permite, muitas vezes, antecipar o cálculo da complexidade para a fase de projeto do algoritmo.
  - Além disso, esse gênero de análise é útil para avaliar as dificuldades intrínsecas da resolução computacional de um problema.

Complexidade e Desempenho de Algoritmos

- O ***desempenho*** de um algoritmo representa o ***esforço computacional*** de execução ***para uma entrada dada***.
- Por outro lado, a ***complexidade*** de um algoritmo reflete o ***esforço para executá-lo*** sobre um conjunto de entradas (digamos, de um dado “tamanho”).
- Assim:
  - A **complexidade pessimista** dá o pior valor: o máximo esforço.
  - A **complexidade média** (ou esperada) dá o valor esperado: a média dos esforços, levando em conta a probabilidade de ocorrência de cada entrada.
  - A **complexidade otimista** considera o melhor caso: aquele cujo esforço seria mínimo.



Medidas de Desempenho e de Complexidade

- As medidas de desempenho consideram o ***tempo*** e o ***espaço*** requeridos por um algoritmo, relacionadas à velocidade e à quantidade de memória, respectivamente.
- A ***complexidade em tempo*** (temporal) mede o tempo de execução.
- A ***complexidade em espaço*** (espacial) usa como medida de desempenho a quantidade de memória necessária para a execução do algoritmo.

Critérios de Complexidade

- O tempo requerido por um algoritmo sobre uma dada entrada pode ser medido pelo ***número de execuções*** de algumas operações.
- Para medir a quantidade de trabalho realizado por um algoritmo, costuma-se escolher uma operação, chamada ***operação fundamental***.
- A operação escolhida como fundamental deve ser tal que a *contagem do número de vezes que ela é executada expresse a quantidade de trabalho do algoritmo*, dispensando outras medidas.
    - Por exemplo, para um algoritmo de ordenação, uma operação fundamental natural é a *comparação* entre elementos quanto à ordem. Para o caso da busca em tabela, uma operação fundamental razoável consiste na comparação, quanto à igualdade, de um elemento da tabela com a chave buscada.

Critérios de Complexidade

- Às vezes, pode ser necessário mais de uma operação fundamental, talvez com pesos diferentes.
- Em qualquer caso, temos uma noção de ***custo*** como um real não negativo.
- Escolhida a noção de custo, consideramos a sequência de passos executados pelo algoritmo com uma entrada dada.
- É contado o ***número de execuções*** das operações fundamentais na execução do algoritmo.
- Assim, obtemos o custo: *o esforço computacional desse algoritmo sobre essa entrada*.

$$d \rightarrow \underbrace{s_0 \rightarrow \cdots \rightarrow s_k \rightarrow \cdots \rightarrow s_n}_{\text{custo}}$$

Desempenho de um Algoritmo

- ***Exemplo***: valor máximo de uma tabela

```
Procedimento Max(tab : tabela)                                  {para máximo de tabela}
    { Entrada: tabela tab : vetor [ 1.. n ] de Elmord.
      Saída: Mx = valor máximo em tab.}

  1. Mx ← tab[1] ;                                              {candidato a máximo}
  2. para i de 2 até n faça                                     {varre tabela: 4}
  3.       se Mx < tab[i] então Mx ← tab[i] fim-se ;            {atualiza Mx}
  4. fim-para ;                                                 {2: tabela varrida}
  5. retorne-saída( Mx )                                        {retorna valor máximo}
  6. fim-Procedimento                                           {fim do algoritmo: máximo de tabela}
```

- Tomemos como operação fundamental a comparação < (no “se” da linha 3).
- Vê-se que Max executa n – 1 vezes a linha 3, fazendo n – 1 comparações.
- Desse modo, o desempenho de Max em qualquer caso é ***n – 1 comparações***.

Desempenho de um Algoritmo

De forma mais geral:

- Dado um algoritmo **a**, considere os conjuntos:
  - D, de seus dados de entrada, e
  - E, de todas as sequências de execuções de operações fundamentais.
- Então, teremos as seguintes funções:
  - ***Execução:*** exec[**a**]: D → E, onde *exec*[**a**](*d*) dá a sequência de execuções de operações fundamentais efetuadas na execução do algoritmo a com entrada $d \in$ D.
  - ***Custo:*** custo: E → $\mathbb{R}_+$, onde *custo*(s) dá o comprimento da sequência $s \in$ E, definido conforme os pesos estabelecidos para as operações fundamentais.
- Agora, podemos definir o desempenho como a composta dessas funções:
  - ***Desempenho***: *desemp*[**a**]: D → $\mathbb{R}_+$, onde *desemp*[**a**](*d*) = *custo*(*exec*[**a**](*d*)) dá o custo (em termos das operações fundamentais) da execução do algoritmo **a** com entrada $d \in$ D.

Complexidade de um Algoritmo

- A complexidade de um algoritmo pode ser obtida condensando-se a informação dada por seu desempenho.
  - Note que a função *desemp*: D → $\mathbb{R}_+$ dá informação sobre cada entrada específica. Em geral, essa informação é demasiado detalhada, e a análise de tal função é bastante difícil.
- Deseja-se condensar a informação dada pela função desempenho em termos das entradas de um tamanho dado. Para tanto, seleciona-se uma função de tamanho.
  - ***Tamanho***: *tam*: D → $\mathbb{N}$, onde *tam*(d) mede o tamanho da entrada $d \in$ D.
  - Por exemplo, para matrizes quadradas, tam(d) dá a ordem de d e, para listas, tam(d) pode ser o comprimento de d.

Complexidade de um Algoritmo

- De posse de tal função *tam*, pode-se pensar em condensar a informação dada pela função *desemp*[**a**]: D → $\mathbb{R}_+$ em uma função *aval*[**a**]: $\mathbb{N}$ → $\mathbb{R}_+$, de modo que para *tam*(d) = *n*: *aval*[**a**](*n*) dá informação sobre *desemp*[**a**](*d*).
  - Essa informação pode ser uma aproximação ou uma ***cota*** (como *desemp*[**a**](*d*) ≤ *aval*[**a**](*n*)).

$$\begin{tikzcd}
D \arrow[r, "desemp"] \arrow[d, "tam"'] & \mathbb{R}_+ \arrow[d, leftrightarrow, "\sim"] \\
\mathbb{N} \arrow[r, "aval"'] & \mathbb{R}_+
\end{tikzcd}$$

Complexidade de um Algoritmo

Complexidade Média

- Considere um algoritmo a que possa receber 100 entradas de tamanho 10: cada entrada d_j apresentando desempenho r_j. Aqui, o desempenho esperado é o valor médio:

$$c_M[a](10) = \frac{r_1 + \ldots + r_{100}}{100}$$

- Nesse caso, considerou-se que cada uma das 100 possíveis entradas tem a *mesma probabilidade* de ocorrer: tomou-se uma ***distribuição uniforme***.

Complexidade de um Algoritmo

Complexidade Média

- De maneira geral, a complexidade média (ou esperada) de um algoritmo **a** (dita $c_M[\mathbf{a}]$ ou $aval_M[\mathbf{a}]$) é o *valor médio de seus desempenhos sobre todas as entradas com tamanho n*, usando-se como peso a *probabilidade de ocorrência* de cada entrada

$$aval_M[a](n) := \sum_{d \in D_n} [\mathrm{prob}(d) \cdot desemp[a](d)]$$

onde prob(d) dá a probabilidade de ocorrer a entrada d no conjunto D_n.

- ***Obs***: Dado $n \in \mathbb{N}$, D_n é o conjunto das entradas com tamanho n:

$$D_n := \{d \in D \mid tam(d) = n\}$$

Complexidade de um Algoritmo

Complexidade Pessimista

- Imagine um algoritmo **a** cujo desempenho sobre cada entrada é o tamanho da entrada. Então, o seu pior desempenho sobre entradas com tamanho até 20 será 20

$$c_P[\mathbf{a}](20) = \text{Máx}\{1, 2, \ldots, 19, 20\} = 20.$$

- A complexidade pessimista de um algoritmo fornece seu desempenho no ***pior caso***: o pior desempenho que se pode esperar. Aqui, pode-se considerar os desempenhos sobre todas as entradas com tamanho *n*.
- A complexidade pessimista (pior caso) de um algoritmo **a** (dita $c_P[\mathbf{a}]$ ou $aval_P[\mathbf{a}]$) é o valor máximo de seus desempenhos sobre todas as entradas com tamanho n

$$aval_P[\mathbf{a}](n) = \text{Máx}\{desemp[\mathbf{a}](d) \in \mathbb{R}_+ \mid d \in D_n\}.$$

Exemplo: Busca Sequencial

```
Procedimento Busca_Seq(ch : Valor, tab : tabela)                {busca sequencial em tabela}
    { Entrada: ch: Valor, tabela tab : vetor [ 1.. n ] de Valor.
      Saída: achou, pos (achou = F se ch não está em tab, caso contrário achou = V e
      tab[pos] = ch).}
1.  i ← 0 ; achou ← F; pos ← 0;                                   { inicialização }
2.  repita                                                        {varre tabela: 5}
3.      i ← i + 1;                                                {incrementa índice}
4.      se tab[i] = ch então achou ← V fim-se                     {compara com chave}
5.  até achou ou i = n;                                           {2: tabela varrida}
6.  se achou então pos ← i fim-se;                                {verifica se achou chave}
7.  retorne-saída( achou, pos )                                   {retorna saída}
8.  fim-Procedimento                                              {fim do algoritmo: busca sequencial em tabela}
```

- O algoritmo Busca_Seq compara a chave ch com cada elemento tab[i] até encontrar um igual ou esgotar a tabela sem encontrar a chave.
- Tomemos como operação fundamental a comparação = (no “se” da linha 4) e como tamanho da entrada o tamanho *n* de tab.

Exemplo: Busca Sequencial

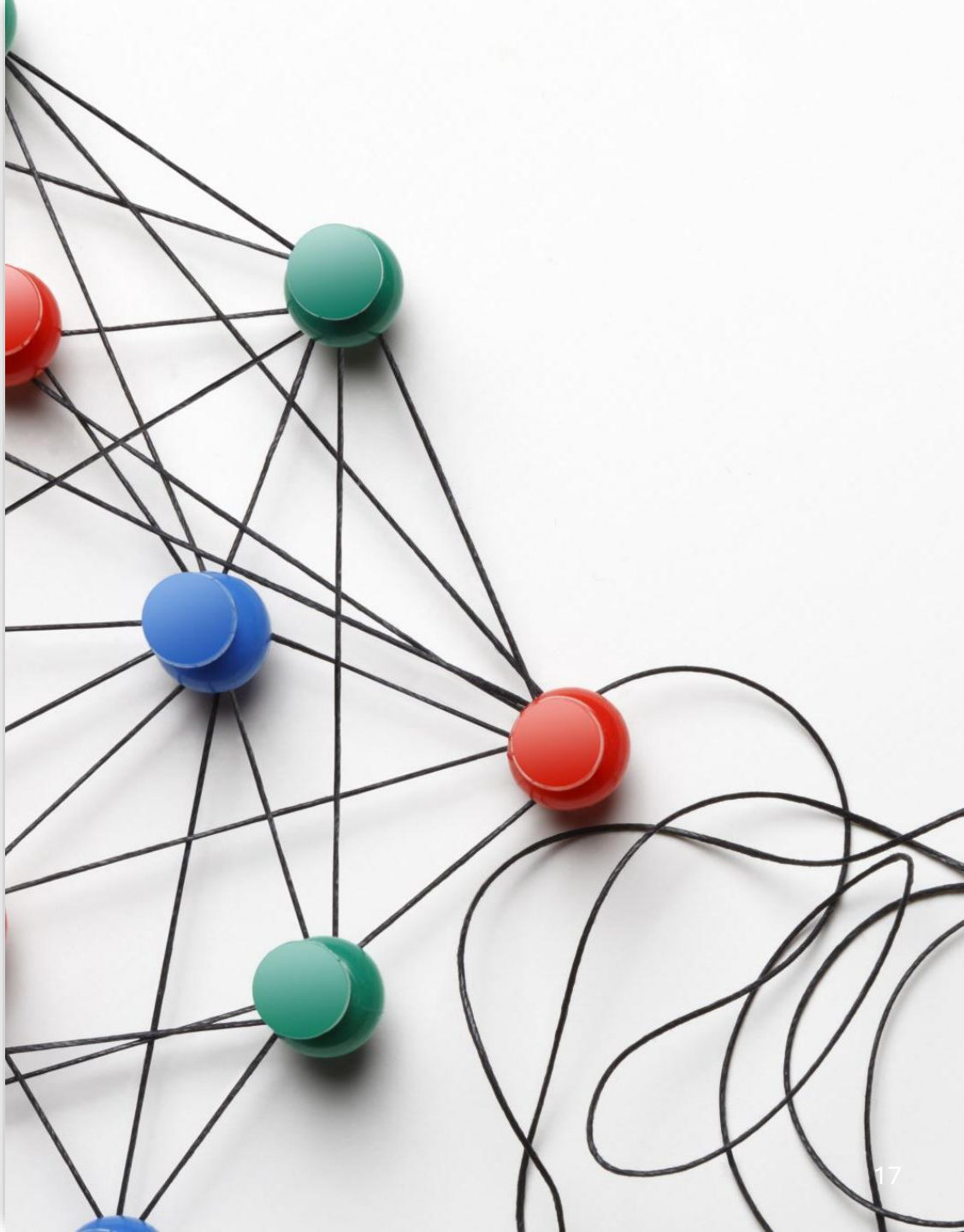
- Assim, a complexidade no pior caso é ***n*** (dizemos: linear no tamanho *n* da entrada).
- Se todos os elementos da tabela forem igualmente prováveis como chave, então o número *médio* de comparações será:

$$\frac{1}{n} \cdot (1 + 2 + 3 + \ldots + n) = \boxed{\frac{1+n}{2}}$$

Complexidade média

Comparação entre Complexidades

- Um problema pode ter mais de um algoritmo para resolvê-lo.
  - Qual deles escolher?
- Em geral, deseja-se selecionar o *melhor*, isto é, o *mais eficiente*.
- Isso envolve comparar as complexidades dos algoritmos.
- Estamos interessados em identificar um ***algoritmo ótimo***.
  - Trata-se de um algoritmo **o** ∈ A tal que *aval*(**o**) é melhor do que *aval*(**a**) para todo *a* ∈ A.



Exercícios

Baseando-se nos conceitos aprendidos até aqui, analise e responda as questões a seguir.

1. Considere dois algoritmos $\mathbf{a}_1$ e $\mathbf{a}_2$ com complexidades $8n^2$ e n^3, respectivamente. Qual o maior valor de n, para o qual o algoritmo $\mathbf{a}_2$ é mais eficiente que o algoritmo $\mathbf{a}_1$?

2. Um algoritmo tem complexidade $2n^2$. Num certo computador, num tempo t, o algoritmo resolve um problema de tamanho 25. Imagine agora que você tem disponível um computador 100 vezes mais rápido. Qual o tamanho máximo de problema que o mesmo algoritmo resolve no mesmo tempo t no computador mais rápido?

3. Considere o mesmo problema anterior para um algoritmo de complexidade 2^n.

4. Suponha que uma empresa utiliza um algoritmo de complexidade n^2 que, em um tempo t, na máquina disponível, resolve um problema de tamanho x. Suponha que o tamanho do problema a ser resolvido aumentou em 20%, mas o tempo de resposta deve ser mantido. Para isso, a empresa pretende trocar a máquina por uma mais rápida. Qual percentual de melhoria no tempo de execução das operações básicas é necessário para atingir sua meta?

5. Suponha que, no problema anterior, ainda se queira reduzir em 50% o tempo de resposta. Qual a melhoria esperada para a nova máquina?

6. São dados um problema a ser resolvido e um tempo máximo de resposta aceitável x. O algoritmo disponível, que tem complexidade 4^n, resolve, nesse tempo, um problema de tamanho máximo n. Entretanto, o tamanho do problema a ser resolvido agora é $2n$. Deseja-se um novo algoritmo que resolva o novo problema no mesmo tempo x. Qual deve ser a complexidade do novo algoritmo?

7. Seja um algoritmo **a** que requer $4n^2 – 3n$ operações para resolver um problema de tamanho n. Deseja-se, mantendo o mesmo tempo de resposta, resolver um problema 30% maior, desenvolvendo um algoritmo mais rápido. Qual deverá ser a complexidade do novo algoritmo?

Até a próxima aula!!